MARA — Multi-party Addressee-aware Real-time Assistant

A Context-Aware Voice Agent for Selective Participation in Group Meetings

---

1. Abstract

MARA is a real-time, audio-only AI agent designed to passively listen to a multi-person meeting and intelligently decide **when to speak and when to stay silent**. Unlike single-user voice assistants, MARA operates inside a multi-speaker conversation where most speech is *not* meant for it. Its core challenge is not transcription or response generation — both are largely solved problems — but **determining whether a given utterance is directed at the agent**, and if so, whether it warrants a response. The project narrows its scope to a well-defined meeting environment (known participants, channel- or name-tagged audio) so that engineering effort is concentrated on the genuinely hard, research-relevant problem: turn-taking and addressee detection in group conversation.

---

2. Motivation & Problem Statement

Most commercial voice assistants (Alexa, Siri, Google Assistant) rely on a wake word and operate in a one-speaker, one-listener context. Meeting assistants like transcription bots (Otter.ai, Fireflies) summarize after the fact but do not participate live. There is a meaningful gap for an agent that can sit *inside* a live multi-person conversation, understand who is talking to whom, and only interject when it is genuinely being asked something — without needing a wake word and without interrupting human-to-human dialogue.

---

3. Defined Scope

To keep the project achievable within a single semester while preserving its core difficulty, the following simplifying assumptions are made:

- **Controlled meeting environment**: the system is designed for a known, fixed set of participants (e.g., a team meeting or class discussion), not an open-world unknown-speaker setting.
- **Speaker identification via channel/name tagging**, not full speaker diarization. Each participant is identified either by a dedicated audio channel (e.g., per-participant microphone/device, similar to Zoom/Teams per-user audio tracks) or a registered name tag at session start. This removes the need for unsupervised speaker embedding/clustering, which is a separate, well-studied ML problem outside the scope of this project.
- **The core deliverable is the decision-making layer**: turn-taking, addressee detection, and intent classification — the modules that decide *whether* and *when* the agent should speak.

- Response generation and text-to-speech use existing, off-the-shelf LLM and TTS APIs rather than custom-trained models.

This reframing keeps the project application-focused and demoable, while its novelty lies in the addressee-detection and decision-fusion logic, not in audio engineering.

---

4. Use Case

Scenario: A team of 4–5 colleagues holds a virtual meeting with MARA present as a silent participant, listening at all times.

1. Two participants discuss a scheduling conflict between themselves. MARA detects this is human-to-human conversation and stays silent.
2. A participant pauses and says, *"Hey MARA, what's the deadline for the Q3 report?"* MARA detects an explicit address, classifies the intent as a question requiring a response, and replies concisely.
3. A participant says, *"Let's just ask the assistant to summarize what we discussed,"* — an implicit, third-person reference to MARA. MARA detects this as directed at it despite no explicit address term, and responds.
4. A participant asks a rhetorical question to the group ("Don't you think that's a bit much?"). MARA correctly classifies this as not requiring a response and stays silent.

This use case demonstrates the three core capabilities that differentiate MARA from a simple voice assistant: **distinguishing agent-directed speech from human-directed speech**, **handling implicit as well as explicit addressing**, and **filtering out rhetorical or irrelevant speech** even when technically addressed.

---

5. System Architecture (Simplified Pipeline)

Stage	Function	Notes
1. Audio Ingestion	Capture per-participant audio channel, VAD, noise reduction	Channel-based, not blind multi-speaker capture
2. Speech Recognition (ASR)	Streaming transcription with turn boundaries	Whisper / AssemblyAI streaming
3. Speaker Tagging	Map channel/name to transcript segment	Simplified — direct mapping, no diarization model needed
4. Context Building	Maintain rolling dialogue history with speaker + timestamp metadata	Sliding window buffer
5. Decision Point (at each pause)	Trigger decision modules	—

Stage	Function	Notes
5A. Turn-Taking Decision	Should the agent speak now?	Context-aware turn-taking model
5B. Addressee Detection	Is this utterance directed at the agent (explicit or implicit)?	**Core novel contribution**
5C. Intent/Dialog Act Classification	Does the utterance require a response (question/command vs. rhetorical/statement)?	Filters false positives
Decision Fusion	Speak = Turn-Taking=YES **AND** Addressee=AGENT **AND** Intent=requires-response	—
6. Response Generation	Retrieve context, generate concise reply via LLM	Off-the-shelf LLM API
7. Speech Synthesis	Convert response to natural speech	Off-the-shelf TTS API

---

6. Objectives

1. Build a real-time audio pipeline that ingests channel-tagged, multi-participant meeting audio and produces a structured, speaker-attributed transcript.
2. Design and train/fine-tune an **addressee detection module** that distinguishes agent-directed speech (explicit and implicit) from human-to-human speech, using the AMI Meeting Corpus addressee annotations as a base dataset.
3. Design a **turn-taking decision module** that determines, at each conversational pause, whether the agent should speak.
4. Design an **intent/dialog-act classification module** that filters out rhetorical or non-actionable utterances even when addressed to the agent.
5. Combine the three modules via a decision-fusion layer and evaluate end-to-end accuracy and latency in live, simulated multi-party meetings.
6. Demonstrate the complete system in a live demo with 3–5 participants.

---

7. Tools & Technologies

Component	Tool/Model Options
Audio ingestion	WebRTC / PyAudio, per-channel capture
ASR	Whisper (large-v3/turbo) or AssemblyAI Streaming
Context buffer	Custom Python buffer / LangChain memory

Component	Tool/Model Options
Turn-taking	Fine-tuned LLM or prompting with reasoning traces, trained on MultiWOZ/AMI/DyLAN
Addressee detection	Fine-tuned BERT/RoBERTa classifier on AMI addressee annotations
Intent/dialog-act	dialogue-act-bert, trained on Switchboard/DAMSL/ISO 24617
Response generation	GPT-4o / Claude / Llama-3 (API)
Speech synthesis	ElevenLabs / Azure Neural TTS / Coqui

---

8. Dataset & Resources

- **AMI Meeting Corpus** — multi-party meetings with addressee annotations (primary dataset for addressee detection and turn-taking).
- **MultiWOZ / DyLAN** — turn-taking dialogue acts.
- **Switchboard / DAMSL** — dialog act classification datasets.

---

9. Evaluation Metrics

- Turn-Taking Accuracy (Speak / Stay Silent)
- Addressee Detection Accuracy (explicit vs. implicit vs. not-addressed)
- Dialog Act Classification F1
- Response Appropriateness (human evaluation)
- End-to-End Latency

---

10. Development Roadmap

1. Build the real-time audio + ASR pipeline with channel-based speaker tagging.
2. Build the rolling context/memory module.
3. Train and evaluate the addressee detection model (core deliverable).
4. Train and evaluate the turn-taking and intent classification models.
5. Implement decision-fusion logic combining all three modules.
6. Integrate response generation and TTS using off-the-shelf APIs.
7. End-to-end evaluation and live multi-party demo.